

Информационные системы в химической технологии

Практическая работа № 12

Изучение технологии контейнеризации приложений на примере Docker Community Edition. Установка Docker, загрузка и запуск контейнеров.

Цель работы

Ознакомиться с основными принципами контейнеризации приложений с использованием Docker Community Edition.

Установить Docker Community Edition на виртуальную машину AstraLinux.
Установить образы приложений из хранилища Docker Hub. Запустить контейнеры в Docker и проверить их работоспособность.

Этапы выполнения работы

Необходимо установить Docker Community Edition, и проверить корректность установки с помощью тестового контейнера **hello_world**.

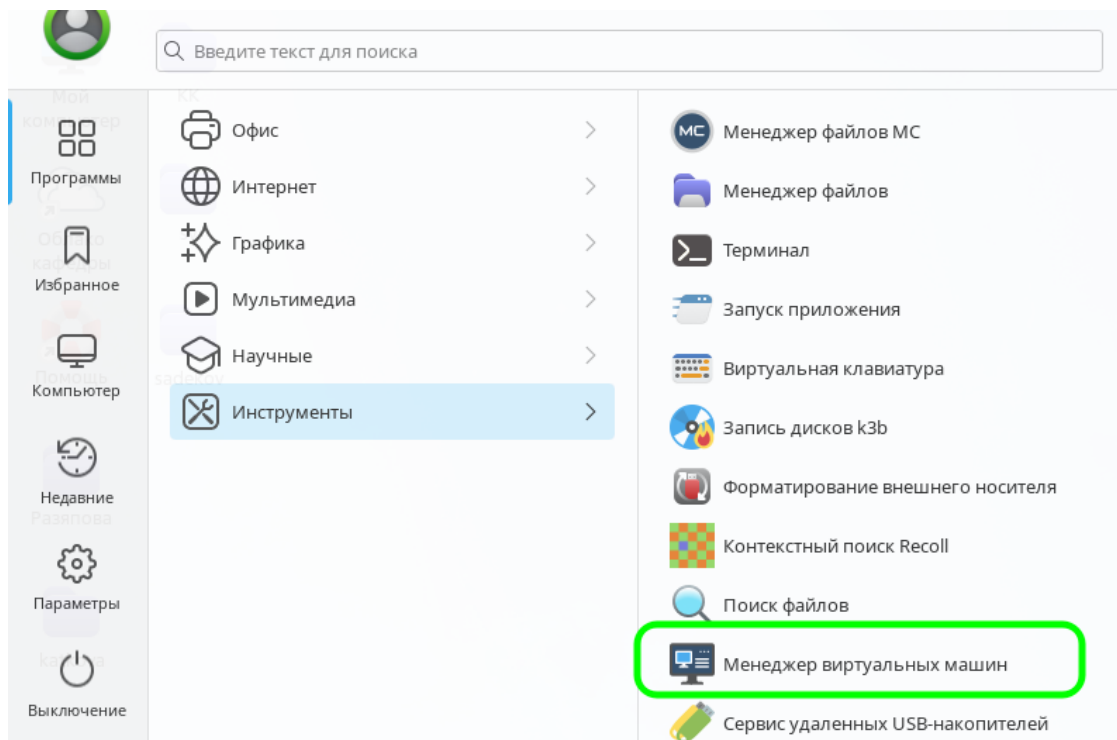
Затем нужно будет установить контейнер **busybox**, и выполнить команды по управлению контейнером.

Последним шагом в Docker необходимо развернуть веб-сервер **linux_tweet_app** с готовым исходным кодом.

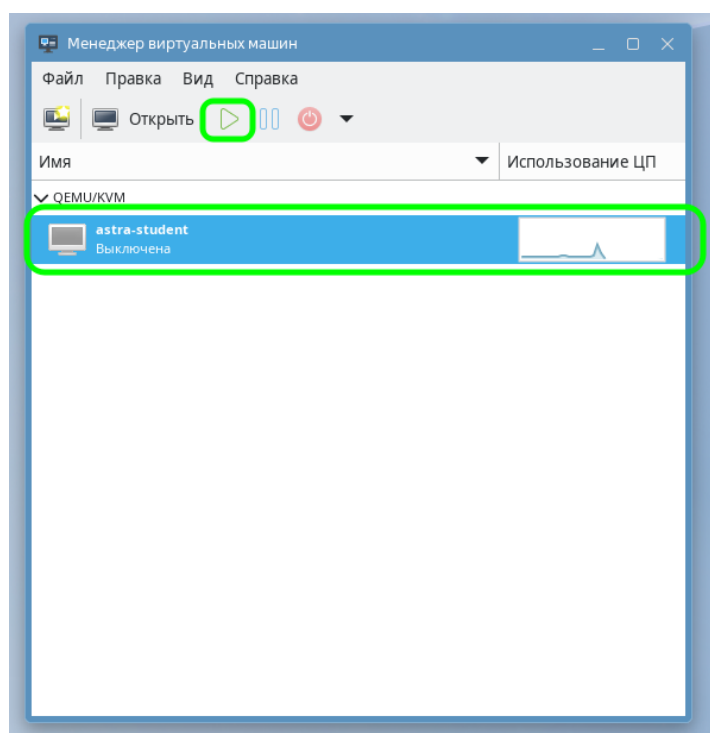
Инструкции

В настоящей работе вы будете осваивать работу с командной строкой операционной системы Astra Linux. Среда с ОС Astra Linux будет представлена в виде виртуальной машины KVM, загруженной на ваш ПК. Для работы вам необходимо получить у преподавателя инструкцию по запуску KVM и запуску виртуальной машины с AstraLinux.

В меню *Пуск* выберите раздел *Инструменты* и запустите программу *Менеджер виртуальных машин*.



В появившемся окне KVM плеера выберите виртуальную машину и нажмите кнопку «Включить» (зеленый треугольник в верхней панели)

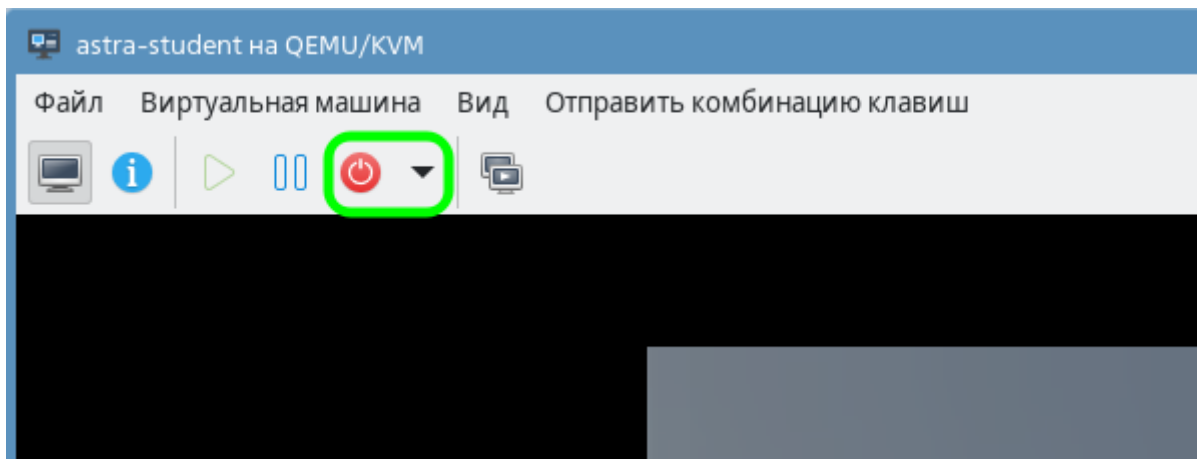


Затем нажмите кнопку «Открыть». Откроется новое окно с запущенной виртуальной машиной.

После запуска виртуальной машины необходимо авторизоваться в системе, введя логин и пароль (помните, что в Linux имена пользователей и пароли регистрозависимые):

Логин: administrator
Пароль: administrator

После окончания работы выключите виртуальную машину, нажав на кнопку «Выключить» в верхней панели.



Шаг 1: Установка Docker Community Edition

Инструкции по установке Docker Community Edition могут отличаться в зависимости от используемой операционной системы. На сайте <https://docs.docker.com/install/> можно найти инструкции по установке.

1. Обновите операционную систему, добавьте gpg ключ для репозитория

```
sudo apt-get update
```

```
sudo apt-get install ca-certificates curl
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -k -fsSL https://download.docker.com/linux/debian/gpg -o  
/etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

2. Подключите официальный репозиторий Docker-CE:

Данная команда добавляет адрес репозитория Docker в файл docker.list

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture) signed-  
by=/etc/apt/keyrings/docker.asc] http://download.docker.com/linux/debian \  
$(. /etc/os-release && echo stretch) stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update
```

3. Установите последнюю версию Docker

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

Если установщик не сможет найти пакет **docker-buildx-plugin**, то этот пакет необходимо установить отдельно, скачав дистрибутив [buildx-plugin](https://download.docker.com/linux/debian/dists/bookworm/pool/stable/amd64/docker-buildx-plugin_0.23.0-1~debian.12~bookworm_amd64.deb) по ссылке https://download.docker.com/linux/debian/dists/bookworm/pool/stable/amd64/docker-buildx-plugin_0.23.0-1~debian.12~bookworm_amd64.deb

Установщик можно запустить из графической оболочки, дважды щелкнув на имени скачанного файла [buildx-v0.23.0.linux-amd64](#).

В процессе установки будет необходимо ввести пароль (administrator)

После установки **docker-buildx-plugin** необходимо повторно запустить установку Docker.

4. Проверьте сервис Docker

```
sudo systemctl status docker
```

Вы должны увидеть вывод, похожий на нижеследующий:

```
docker.service - Docker Application Container Engine  
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled;  
vendor preset: disabled)  
   Active: active (running) since Tue 2016-02-02 18:13:42 EST; 2  
days ago  
     Docs: https://docs.docker.com  
    Main PID: 16915 (docker)  
     Memory: 14.7M  
CGroup: /system.slice/docker.service  
        └─16915 /usr/bin/docker daemon -H fd://  
           └─20162 docker-proxy -proto tcp -host-ip 0.0.0.0 -  
host-port 3375 -container-ip 172.17.0.2 -container-port 2375
```

*Запустите сервис, если он не запущен

```
sudo systemctl enable docker  
sudo systemctl start docker
```

1. Проверьте, что Docker корректно установлен, и определите версию:

```
sudo docker version  
sudo docker info
```

2. Запустите контейнер **hello-world**

```
sudo docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally  
latest: Pulling from library/hello-world  
1b930d010525: Pull complete  
Digest:  
sha256:b8ba256769a0ac28dd126d584e0a2011cd2877f3f76e093a7ae560f2a53  
01c00  
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!  
This message shows that your installation appears to be working  
correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client,
which sent it
to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

Демон Docker использует системный порт TCP для работы, поэтому для использования Docker требуются привилегии root. В качестве альтернативы можно создать группу docker и добавить текущего пользователя в нее.

1. (Опционально) Добавьте текущего пользователя в группу **docker**
 - a. Создайте группу **docker** (если не была создана автоматически)

```
sudo groupadd docker
```

- b. Добавьте текущего пользователя в группу **docker**.

```
sudo usermod -aG docker administrator
```

- c. Повторно запустите терминал, чтобы активировать Изменения.

```
exit
```

- d. Запустите контейнер **hello-world** еще раз, не используйте команду **sudo**.

Шаг 2: Запуск и управление контейнерами

1. Запустите контейнер **busybox**, непрерывно обращающийся к сетевому ресурсу

```
docker run busybox ping google.com
```

```
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
7c9d20b9b6cd: Pull complete
Digest:
sha256:fe301db49df08c384001ed752dff6d52b4305a73a7f608f2152804
8e8a08b51e
Status: Downloaded newer image for busybox:latest
PING google.com (172.217.7.206): 56 data bytes
64 bytes from 172.217.7.206: seq=0 ttl=51 time=0.870 ms
...
```

2. В другом терминале выведите список работающих контейнеров

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
4ce165bfc38e	busybox	"ping google.com"	About a minute ago
Up			

- Остановите контейнер с помощью мягкой остановки. Вам следует использовать ID контейнера из предыдущего листинга. Обратите внимание на задержку в 10 секунд между запуском команды и остановкой контейнера. Она вызвана тем, что процесс ping игнорирует сигнал SIGTERM.

```
docker stop 4ce165bfc38e
```

- Запустите контейнер еще раз и остановите его с помощью команды **docker**. Обратите внимание, что повторная загрузка образа уже не требуется, т.к. он сохранен в локальный кэш узла.
- Для запуска контейнера в фоновом режиме используйте ключ **--detach**

```
docker run --detach busybox ping google.com
```

```
8a61e1caf1aa0bb47928a8ebb1fc9a28acd61b0c2ccfa3cc478deb02b20fa04
```

```
docker kill 8a61e1caf1aa
```

- Выведите список остановленных контейнеров

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED
d65b098c715d	hello-world	"/hello"	21 minutes ago
4ce165bfc38e	busybox	"ping google.com"	About a minute ago
8a61e1caf1aa	busybox	"ping google.com"	About a minute ago
Exited (0)			
Exited (0)			
Exited (0)			

- Просмотрите журнал работы одного из контейнеров.

```
docker logs 8a61e1caf1aa
```

```
PING google.com (172.217.7.206): 56 data bytes
64 bytes from 172.217.7.206: seq=0 ttl=51 time=0.917 ms
```

```
--- google.com ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 0.917/0.917/0.917 ms
```

8. Для диагностики иногда недостаточно информации в журнале. Запустите контейнер и подключитесь к командной строке внутри контейнера.

```
docker run busybox ping google.com
```

- a. В другом терминале запустите команду **docker exec**
docker exec -it a63edf783549 sh

```
/ #
```

- b. Внутри контейнера выведите список процессов и список файлов

```
docker exec -it a63edf783549 sh
```

```
/ # ps -ef
PID    USER      TIME  COMMAND
   1   root         0:00 ping google.com
   6   root         0:00 sh
  11   root         0:00 ps -ef
/ #
/ # ls
bin    dev     etc     home    proc   root    sys     tmp     usr     var
/ #
/ # id
uid=0(root) gid=0(root) groups=10(wheel)
Ctrl + D
```

Шаг 3. Docker. От исходного кода до контейнера

Разверните веб-сервер **linux_tweet_app** с готовым исходным кодом.

1. Проверьте пакет для работы с системой контроля версий **git**:

```
sudo git --version
```

*Если **git** не установлен, сделайте это:

```
sudo apt install git
```


2. Загрузите исходный код и файл сборки образа из репозитория Git

```
git clone https://github.com/dockeramples/linux_tweet_app
```

3. Перейдите в каталог **linux_tweet_app** и соберите образ веб-сервера. Файл сборки образа Dockerfile загружен вместе с исходным кодом

```
cd linux_tweet_app/
```

```
docker build -t linux_tweet_app .
```

В конце команды стоит точка, задающая текущий каталог в качестве целевого каталога !!

4. Запустите контейнер

- а. Укажите порт хоста, который должен соответствовать порту 80 внутри контейнера

```
docker container run --detach -p 8080:80 linux_tweet_app
```

```
8a61e1caf1aa14bb5393c6166fc7ed40cd912994a381db84f8bedfa3e717f2e9
```

- б. Подключитесь к веб-серверу `http://IP_node:8080`



5. Остановите запущенный контейнер

```
docker kill <ID_контейнера>
```

6. Очистите систему

```
docker system prune
```

WARNING! This will remove:

- all stopped containers
- all networks not used by at least one container

- all dangling images
- all build cache

Are you sure you want to continue? [y/N] y

Deleted Containers:

d65b098c715d88c042b632f7b477857664f6035258e25f155912f1d7afad4f34
4ce165bfc38e7e3baab95b49a0bb47928a8ebb1fc9a28acd61b0c2ccfa3cc423
8a61e1caf1aabfe8df48e77a62469da9a4697f8b6a2b5448495cc32c24d6e451
8a61e1caf1aa14bb5393c6166fc7ed40cd912994a381db84f8bedfa3e717f2e9

Total reclaimed space: 355.7kB